



Workpath SDK

Copy Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1	Introduction	3
1.1	Document conventions	3
2	Copy Sample	4
2.1	Copy Sample on a device	5
3	Initializing SDK.....	6
4	Integrating with copy functions.....	8
4.1	Retrieving Capabilities	8
4.1.1	Needs of Capabilities	8
4.1.2	Reading Capabilities	9
4.1.3	Getting defaults	10
4.2	CopyAttributes Creation.....	11
4.3	Submitting a job.....	13
4.3.1	Creating Job Attributes for Copy	13
4.3.1.1	Creating StoreCopyJob	15
4.3.2	Requesting a job	17
4.4	Monitoring Job.....	18
4.5	Releasing stored Job	20
4.6	Deleting stored Job	21
4.7	Enumerating stored Job	22
5	Legal notice	23
6	Support.....	24

1 Introduction

Copy Sample shows use case scenario how to interact with device for copying.

1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- Name - Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses or bold (for example, `function()`, **function()**).

2 Copy Sample

Copy Sample shows how to:

1. Interact with Copy Functions. This calls the device api to:
 - Retrieve its capabilities
 - Build attributes
 - Submit copy jobs
 - Normal copy or Store copy
2. Interact with Job Manager
 - Call **JobService** to monitor the current job
 - Handle **JobService.AbstractJobletObserver** callbacks

2.1 Copy Sample on a device



A main activity of Copy Sample on a device

3 Initializing SDK

Application requires to initialize the SDK services when starts with an activity for accessing the SDK services. The `initialize()` method supports this.

The sample shows how to initialize SDK.

1. Call `initialize()` method of Workpath SDK on resume event, and clear it on pause event. It is for calling initialization when an activity is switched. Refer to `MainActivity.java`.

```

1. @Override
2. protected void onResume() {
3.     super.onResume();
4.
5.     mCopyConfigureFragment = new CopyConfigureFragment();
6.     mStoreJobFragment = new StoreJobFragment();
7.
8.     replaceFragment(mCopyConfigureFragment);
9.     mTabLayout.getTabAt(0).select();
10.    // Register JobObserver to receive job state callbacks
11.    mJobObserver.register(this);
12.
13.    mContainer.setEnabled(false);
14.
15.    mInitializationTask = new InitializationTask(this);
16.    mInitializationTask.execute();
17. }
18.
19. @Override
20. protected void onPause() {
21.     super.onPause();
22.
23.     mInitializationTask.cancel(true);
24.     mInitializationTask = null;
25.
26.     // Unregister ContentObservers
27.     mJobObserver.unregister(this);
28.
29.     if (mAlertDialog != null) {
30.         mAlertDialog.dismiss();
31.         mAlertDialog = null;
32.     }
33.
34.     if (mSnackBar != null) {
35.         mSnackBar.dismiss();
36.         mSnackBar = null;
37.     }
38. }

```

Initialization call in Activity Resume

2. Make a task and call `initialize()` method with catching exceptions. If Workpath SDK is not initialized, sdk exception or security exception will be thrown. Refer to `InitializationTask.java`.

```

1. @Override
2. protected InitStatus doInBackground(final Void... params) {
3.     InitStatus status = InitStatus.NO_ERROR;
4.
5.     try {
6.         // initialize Workpath SDK
7.         Workpath.getInstance().initialize(mContextRef.get());
8.     } catch (SdkUnsupportedException sue) {
9.         Log.e(TAG, "SDK is not supported!", sue);
10.        mException = sue;
11.        status = InitStatus.INIT_EXCEPTION;
12.    } catch (SecurityException se) {
13.        Log.e(TAG, "Security exception!", se);
14.        mException = se;
15.        status = InitStatus.INIT_EXCEPTION;
16.    }
17.
18.    // Check if CopyService is supported
19.    if (status == InitStatus.NO_ERROR
20.        && (!CopierService.isSupported(mContextRef.get())
21.            || !JobService.isSupported(mContextRef.get()))) {
22.        // CopyService is not supported on this device
23.        status = InitStatus.NOT_SUPPORTED;
24.    }
25.
26.    return status;
27. }

```

Exception handling by initializationTask

4 Integrating with copy functions

Copy Sample explains how to build copy attributes and submit a job. Before starting a copy, capabilities should be checked.

4.1 Retrieving Capabilities

4.1.1 Needs of Capabilities

The capability of copy service attributes is retrieved to ensure that the user is selecting one of the allowed values. Since the capability of copy attribute can vary among devices, it is very important to prevent the user from selecting unsupported attribute values as you develop your app.

4.1.2 Reading Capabilities

Reading capability requires interaction with a device and must not be invoked from the UI thread. Therefore, `MainActivity` executes `LoadCapabilitiesTask()`, which then calls the `requestCaps()` method obtaining values from `getCapabilities` of `CopierService` to read the capabilities.

The `CopierService.getCapabilities(context, result)` will return any errors that exist in `result` argument.

```

1.  /**
2.   * Executes request for capabilities from CopierService.
3.   *
4.   * @param context {@link Context}
5.   * @return {@link com.hp.workpath.api.copier.CopyAttributesCaps}
6.   */
7.  public CopyAttributesCaps requestCaps(final Context context, Result result) {
8.      if (result == null) {
9.          result = new Result();
10.     }
11.
12.     // cache capabilities for building CopyAttributes
13.     mCapabilities = CopierService.getCapabilities(context, result);
14.
15.     if (result.getCode() == Result.RESULT_OK && mCapabilities != null) {
16.         showResult("Caps=" + Logger.build(mCapabilities));
17.     } else {
18.         showResult("CopierService.getCapabilities(): ", result);
19.     }
20.
21.     return mCapabilities;
22. }

```

Loading capabilities

4.1.3 Getting defaults

The `CopierService.getDefault(context, result)` returns default attributes for copy from a device.

```
1. @Override
2. protected CopyAttributes doInBackground(final Void... params) {
3.     return CopierService.getDefault(mContextRef.get(), result);
4. }
5.
6. @Override
7. protected void onPostExecute(final CopyAttributes defaults) {
8.     super.onPostExecute(defaults);
9.
10.    if (defaults != null && result.getCode() == Result.RESULT_OK) {
11.        Log.i(TAG, "Defaults=" + LogUtil.getCopyAttributesToString(defaults));
12.        mContextRef.get().showResult(mContextRef.get().getString(R.string.succeed));
13.        mFragment.get().setDefaultCopyAttributes(defaults);
14.    } else {
15.        mContextRef.get().showResult("CopierService.getDefault()", result);
16.    }
17. }
```

Getting defaults

4.2 CopyAttributes Creation

As in Copy Sample, the *PreferenceFragment* contains a variety of Sample attributes to be set manually. Please see [CopyConfigureFragment.java](#).

```

1.  /**
2.   * Simple {@link PreferenceFragmentCompat} to set Copy Attributes and save into preferences.
3.   */
4.  public final class CopyConfigureFragment extends PreferenceFragmentCompat implements
5.      SharedPreferences.OnSharedPreferenceChangeListener {
6.
7.      // Preferences keys for CopyAttributes
8.      public static final String PREF_COPIES = "pref_copies";
9.      public static final String PREF_COLOR_MODE = "pref_colorMode";
10.     public static final String PREF_ORIENTATION = "pref_orientation";
11.     public static final String PREF_SCAN_DUPLEX_MODE = "pref_scanDuplexMode";
12.     public static final String PREF_PRINT_DUPLEX_MODE = "pref_printDuplexMode";
13.     public static final String PREF_SCAN_SIZE = "pref_scanSize";
14.     public static final String PREF_SCAN_CUSTOM_LENGTH = "pref_scan_customLength";
15.     public static final String PREF_SCAN_CUSTOM_WIDTH = "pref_scan_customWidth";
16.     public static final String PREF_PRINT_SIZE = "pref_printSize";
17.     public static final String PREF_PRINT_CUSTOM_LENGTH = "pref_print_customLength";
18.     public static final String PREF_PRINT_CUSTOM_WIDTH = "pref_print_customWidth";
19.     public static final String PREF_COPY_PREVIEW = "pref_copyPreview";
20.     public static final String PREF_SCAN_SOURCE = "pref_scanSource";
21.     public static final String PREF_BACKGROUND_CLEANUP = "pref_backgroundCleanup";
22.     public static final String PREF_CONTRAST_ADJUSTMENT = "pref_contrastAdjustment";
23.     public static final String PREF_DARKNESS_ADJUSTMENT = "pref_darknessAdjustment";
24.     public static final String PREF_SHARPNESS_ADJUSTMENT = "pref_sharpnessAdjustment";
25.     public static final String PREF_TEXT_GRAPHICS_OPTIMIZATION = "pref_textGraphicsOpti
    mization";
26.     public static final String PREF_COLLATE_MODE = "pref_collateMode";
27.     public static final String PREF_PAPER_SOURCE = "pref_paperSource";
28.     public static final String PREF_PAPER_TYPE = "pref_paperType";
29.     public static final String PREF_SCALE_MODE = "pref_scaleMode";
30.     public static final String PREF_SCALE_PERCENT = "pref_scalePercent";
31.     public static final String PREF_NUMBER_UP_MODE = "pref_numberUpMode";
32.     public static final String PREF_NUMBER_UP_DIRECTION = "pref_numberUpDirection";
33.     public static final String PREF_JOB_ASSEMBLY_MODE = "pref_jobAssemblyMode";
34.     public static final String PREF_JOB_EXECUTION_MODE = "pref_jobExecutionMode";
35.     public static final String PREF_STORE_JOB_NAME = "pref_storedJobName";
36.     public static final String PREF_STORE_JOB_FOLDER_NAME = "pref_storedJobFolderName";
37.     public static final String PREF_STORE_DELETE_ON_POWER = "pref_storedDeleteOnPower";
38.     public static final String PREF_STORE_DELETE_ON_RELEASE = "pref_storedDeleteOnRelea
    se";
39.     public static final String PREF_STORE_JOB_PASSWORD_TYPE = "pref_storedJobPasswordTy
    pe";
40.     public static final String PREF_STORE_JOB_PASSWORD = "pref_storedJobPassword";
41.     public static final String PREF_BASE_ATTRIBUTES_CATEGORY = "base_attributes_categor
    y";
42.     public static final String PREF_DESTINATION_STORE_CATEGORY = "destination_store_cat
    egory";
43.     public static final String PREF_MONITOR_JOB = "pref_monitorJob";
44.     public static final String PREF_SHOW_JOB_PROGRESS = "pref_showJobProgress";
45.     public static final String PREF_SETTINGS_UI = "pref_settingsUi";
46.     public static final String CURRENT_JOB_ID = "pref_currentJobId";
47.     public static final String PREF_OUTPUT_BIN = "pref_outputBin";

```

```

48.     public static final String PREF_PROGRESS_DIALOG_MODE = "pref_progressDialogMode";
49.     public static final String PREF_ERASE_MARGIN_UNIT = "pref_eraseMarginUnit";
50.     public static final String PREF_ERASE_BACK_MARGIN = "pref_eraseBackMargin";
51.     public static final String PREF_ERASE_BACK_LEFT_MARGIN = "pref_eraseBackMarginLeft"
    ;
52.     public static final String PREF_ERASE_BACK_TOP_MARGIN = "pref_eraseBackMarginTop";
53.     public static final String PREF_ERASE_BACK_RIGHT_MARGIN
    = "pref_eraseBackMarginRight";
54.     public static final String PREF_ERASE_BACK_BOTTOM_MARGIN = "pref_eraseBackMarginBot
    tom";
55.     public static final String PREF_ERASE_FRONT_MARGIN = "pref_eraseFrontMargin";
56.     public static final String PREF_ERASE_FRONT_LEFT_MARGIN
    = "pref_eraseFrontMarginLeft";
57.     public static final String PREF_ERASE_FRONT_TOP_MARGIN
    = "pref_eraseFrontMarginTop";
58.     public static final String PREF_ERASE_FRONT_RIGHT_MARGIN
    = "pref_eraseFrontMarginRight";
59.     public static final String PREF_ERASE_FRONT_BOTTOM_MARGIN
    = "pref_eraseFrontMarginBottom";
60.     public static final String PREF_CAPTURE_MODE = "pref_captureMode";
61.     public static final String PREF_IMAGE_SHIFT_REDUCE_TO_FIT
    = "pref_imageShiftReduceToFit";
62.     public static final String PREF_IMAGE_SHIFT_UNITS = "pref_imageShiftUnits";
63.     public static final String PREF_IMAGE_SHIFT_FRONT = "pref_imageShiftFront";
64.     public static final String PREF_IMAGE_SHIFT_BACK = "pref_imageShiftBack";
65.     public static final String PREF_IMAGE_SHIFT_X_FRONT = "pref_imageShiftFrontxShift";
66.     public static final String PREF_IMAGE_SHIFT_Y_FRONT = "pref_imageShiftFrontyShift";
67.     public static final String PREF_IMAGE_SHIFT_X_BACK = "pref_imageShiftBackxShift";
68.     public static final String PREF_IMAGE_SHIFT_Y_BACK = "pref_imageShiftBackyShift";
69.     public static final String PREF_BOOKLET_BORDERS_EACH_PAGE
    = "pref_bookletBordersEachPage";
70.     public static final String PREF_BOOKLET_FINISHING_OPTION
    = "pref_bookletFinishingOption";
71.     public static final String PREF_BOOKLET_FORMAT = "pref_bookletFormat";
72.     public static final String PREF_FOLD_MODE = "pref_foldMode";
73.     public static final String PREF_STAPLE_MODE = "pref_stapleMode";
74.     public static final String PREF_PUNCH_MODE = "pref_punchMode";
75.     public static final String PREF_WATERMARK_ROTATION = "pref_watermarkRotate";
76.     public static final String PREF_WATERMARK_TEXT = "pref_watermarkText";
77.     public static final String PREF_WATERMARK_DARKNESS = "pref_watermarkDarkness";
78.     public static final String PREF_WATERMARK_TEXT_SIZE = "pref_watermarkTextSize";
79.     public static final String PREF_WATERMARK_TYPE = "pref_watermarkType";
80.     public static final String PREF_WATERMARK_MESSAGE_TYPE
    = "pref_watermarkMessageType";
81.     public static final String PREF_WATERMARK_BACKGROUND_PATTERN
    = "pref_watermarkBackgroundPattern";
82.     public static final String PREF_WATERMARK_TRANSPARENCY
    = "pref_watermarkTransparency";
83.     public static final String PREF_WATERMARK_BACKGROUND_COLOR
    = "pref_watermarkBackgroundColor";
84.     public static final String PREF_WATERMARK_FONT = "pref_watermarkFont";
85.     public static final String PREF_WATERMARK_TEXT_COLOR = "pref_watermarkTextColor";
86.     public static final String PREF_WATERMARK_ONLY_FIRST_PAGE
    = "pref_watermarkOnlyFirstPage";
87.     public static final String PREF_STAMP = "pref_stamp";

```

Configuring copy attributes using preferenceFragment

4.3 Submitting a job

Copy job submission involves:

- Creating job attributes with Builders
- Submitting the copy job
- Observing the job events and starting monitoring of job state.

4.3.1 Creating Job Attributes for Copy

CopyAttributes class provides flexibility in terms of the type of builder used to create copy attributes. The simple method of normal copy for scanning files and printing is called with normal copy **Builder**.

Sample code shows how to build attributes.

```

1.  switch (jobExecutionMode) {
2.      case NORMAL:
3.          return new CopyAttributes.CopyBuilder()
4.              .setColorMode(colorMode)
5.              .setOrientation(orientation)
6.              .setScanDuplex(scanDuplex)
7.              .setScanSize(scanSize)
8.              .setScanCustomLength(scanCustomLength)
9.              .setScanCustomWidth(scanCustomWidth)
10.             .setScanSource(scanSource)
11.             .setCopyPreview(copyPreview)
12.             .setBackgroundCleanup(backgroundCleanup)
13.             .setContrastAdjustment(contrastAdjustment)
14.             .setDarknessAdjustment(darknessAdjustment)
15.             .setSharpnessAdjustment(sharpnessAdjustment)
16.             .setPrintDuplex(printDuplex)
17.             .setPrintSize(printSize)
18.             .setPrintCustomLength(printCustomLength)
19.             .setPrintCustomWidth(printCustomWidth)
20.             .setCopies(copies)
21.             .setCollateMode(collateMode)
22.             .setPaperSource(paperSource)
23.             .setPaperType(paperType)
24.             .setScaleMode(scaleMode)
25.             .setScalePercent(scalePercent)
26.             .setTextGraphicsOptimization(textGraphicsOptimization)
27.             .setJobAssemblyMode(jobAssemblyMode)
28.             .setNumberUpMode(numberUpMode)
29.             .setNumberUpDirection(numberUpDirection)
30.             .setOutputBin(outputBin)
31.             .setProgressDialogMode(progressDialogMode)
32.             .setEraseMarginUnit(eraseMarginUnit)
33.             .setEraseBackMargin(backMargin)
34.             .setEraseFrontMargin(frontMargin)
35.             .setCaptureMode(captureMode)
36.             .setImageShiftReduceToFit(imageShiftReduceToFit)
37.             .setImageShiftUnits(imageShiftUnits)
38.             .setImageShiftFront(imageShiftFront)
39.             .setImageShiftBack(imageShiftBack)

```

```
40.         .setBookletBordersEachPage(bookletBordersEachPage)
41.         .setBookletFinishingOption(bookletFinishingOption)
42.         .setBookletFormat(bookletFormat)
43.         .setFoldMode(foldMode)
44.         .setStapleOption(stapleOption)
45.         .setPunchMode(punchMode)
46.         .setWatermarkDarkness(watermarkDarkness)
47.         .setWatermarktextSize(watermarkTextSize)
48.         .setWatermarkTransparency(watermarkTransparency)
49.         .setWatermarkFont(watermarkFont)
50.         .setWatermarkBackgroundColor(watermarkBackgroundColor)
51.         .setWatermarkTextColor(watermarkTextColor)
52.         .setWatermarkOnlyFirstPage(watermarkOnlyFirstPage)
53.         .setWatermarkText(watermarkText)
54.         .setWatermarkRotate45(watermarkRotate45)
55.         .setWatermarkType(watermarkType)
56.         .setWatermarkBackgroundPattern(watermarkBackgroundPattern)
57.         .setWatermarkMessageType(watermarkMessageType)
58.         .build(capabilities);
```

Building attributes with Builder

4.3.1.1 Creating StoreCopyJob

`CopyAttributes` class provides `StoreCopyBuilder()` for storing job after scanning.

Sample code shows how to build attributes.

```

1. CopyAttributes.StoreCopyBuilder storeCopyBuilder = new CopyAttributes.StoreCopyBuilder(
   );
2.
3. if (storeJobPasswordType != JobCredentialsAttributes.PasswordType.NONE) {
4.     JobCredentialsAttributes jobCredentialsAttributes = new JobCredentialsAttributes.Bu
       ilder()
5.         .setPasswordType(storeJobPasswordType)
6.         .setPassword(storeJobPassword)
7.         .build();
8.
9.     storeCopyBuilder.setStoreJobCredentials(jobCredentialsAttributes);
10. }
11.
12. return storeCopyBuilder
13.     .setStoreJobName(storeJobName)
14.     .setStoreJobFolderName(storeJobFolderName)
15.     .setRetentionModeOnPowerCycle(deleteOnPower)
16.     .setRetentionModeOnRelease(deleteOnRelease)
17.     .setColorMode(colorMode)
18.     .setOrientation(orientation)
19.     .setScanDuplex(scanDuplex)
20.     .setScanSize(scanSize)
21.     .setScanCustomLength(scanCustomLength)
22.     .setScanCustomWidth(scanCustomWidth)
23.     .setScanSource(scanSource)
24.     .setCopyPreview(copyPreview)
25.     .setBackgroundCleanup(backgroundCleanup)
26.     .setContrastAdjustment(contrastAdjustment)
27.     .setDarknessAdjustment(darknessAdjustment)
28.     .setSharpnessAdjustment(sharpnessAdjustment)
29.     .setPrintDuplex(printDuplex)
30.     .setPrintSize(printSize)
31.     .setPrintCustomLength(printCustomLength)
32.     .setPrintCustomWidth(printCustomWidth)
33.     .setCopies(copies)
34.     .setCollateMode(collateMode)
35.     .setPaperSource(paperSource)
36.     .setPaperType(paperType)
37.     .setScaleMode(scaleMode)
38.     .setScalePercent(scalePercent)
39.     .setTextGraphicsOptimization(textGraphicsOptimization)
40.     .setJobAssemblyMode(jobAssemblyMode)
41.     .setNumberUpMode(numberUpMode)
42.     .setNumberUpDirection(numberUpDirection)
43.     .setOutputBin(outputBin)
44.     .setProgressDialogMode(progressDialogMode)
45.     .setEraseMarginUnit(eraseMarginUnit)
46.     .setEraseBackMargin(backMargin)
47.     .setEraseFrontMargin(frontMargin)
48.     .setCaptureMode(captureMode)
49.     .setImageShiftReduceToFit(imageShiftReduceToFit)
50.     .setImageShiftUnits(imageShiftUnits)
51.     .setImageShiftFront(imageShiftFront)
52.     .setImageShiftBack(imageShiftBack)

```

```
53.         .setBookletBordersEachPage(bookletBordersEachPage)
54.         .setBookletFinishingOption(bookletFinishingOption)
55.         .setBookletFormat(bookletFormat)
56.         .setFoldMode(foldMode)
57.         .setStapleOption(stapleOption)
58.         .setPunchMode(punchMode)
59.         .setWatermarkDarkness(watermarkDarkness)
60.         .setWatermarktextSize(watermarkTextSize)
61.         .setWatermarkTransparency(watermarkTransparency)
62.         .setWatermarkFont(watermarkFont)
63.         .setWatermarkBackgroundColor(watermarkBackgroundColor)
64.         .setWatermarkTextColor(watermarkTextColor)
65.         .setWatermarkOnlyFirstPage(watermarkOnlyFirstPage)
66.         .setWatermarkText(watermarkText)
67.         .setWatermarkRotate45(watermarkRotate45)
68.         .setWatermarkType(watermarkType)
69.         .setWatermarkBackgroundPattern(watermarkBackgroundPattern)
70.         .setWatermarkMessageType(watermarkMessageType)
71.         .build(capabilities);
```

Building attributes with Builder

4.3.2 Requesting a job

Sample shows how an application creates copy job and checks job status. Main method for submitting a job is `CopierService.submit()`. This method allows to put `CopyletAttributes` as a parameter to show embedded settings dialog.

The following code shows how to implement it with `CopyletAttributes`.

```
1. final CopyletAttributes taskAttribs = new CopyletAttributes.Builder()
2.     .setShowSettingsUi(settingsUi)
3.     .build();
4.
5. // Submit the job
6. final String rid = CopierService.submit(activity, attributes, taskAttribs);
7.
8. Log.i(TAG, "Job submitted with rid = " + rid);
```

Submitting a copy job with attributes

4.4 Monitoring Job

Callbacks for Copy job are received via `JobService.AbstractJobletObserver`. Copy Sample extends this class to obtain callbacks and the state of job. Registration and unregistration of the observer are done by `OnResume()` / `OnPause()` respectively.

`MainActivity` contains `JobObserver` implementation:

```

1.  /**
2.   * Observer for submitted job
3.   */
4.  private class JobObserver extends JobService.AbstractJobletObserver {
5.      public JobObserver(final Handler handler) {
6.          super(handler);
7.      }
8.
9.      @Override
10.     public void onProgress(final String rid, final JobInfo jobInfo) {
11.         Log.i(TAG, "onProgress: Received rid=" + rid);
12.         Log.i(TAG, "JobInfo=" + LogUtil.getJobInfoToString(jobInfo));
13.         if (rid.equals(mRid)) {
14.             if (mJobId == null) {
15.                 if (jobInfo.getJobId() != null) {
16.                     mJobId = jobInfo.getJobId();
17.
18.                     Log.i(TAG, "Received jobId=" + mJobId);
19.                     showSnackBar(getString(R.string.job_id, mJobId));
20.
21.                     final SharedPreferences prefs = PreferenceManager.getDefaultSharedPreferences(
22.                         references(getApplicationContext()));
23.                     final boolean monitorJob = prefs.getBoolean(CopyConfigureFragment.P
24.                         REF_MONITOR_JOB, true);
25.
26.                     if (monitorJob) {
27.                         prefs.edit().putString(CopyConfigureFragment.CURRENT_JOB_ID, mJ
28.                             obId).apply();
29.
30.                         final boolean showProgress =
31.                             prefs.getBoolean(CopyConfigureFragment.PREF_SHOW_JOB_PR
32.                                 OGRESS, true);
33.
34.                         // Monitor the job completion
35.                         final JobletAttributes taskAttributes =
36.                             new JobletAttributes.Builder().setShowUi(showProgress).
37.                             build();
38.
39.                         final Intent intent = new Intent(getApplicationContext(), JobCo
40.                             mpleteReceiver.class);
41.                         intent.setAction(ACTION_COPY_COMPLETED);
42.                         intent.putExtra(JobCompleteReceiver.RID_EXTRA, rid);
43.                         intent.putExtra(JobCompleteReceiver.JOB_ID_EXTRA, mJobId);
44.
45.                         final String jrid = JobService.monitorJobInForeground(MainActiv
46.                             ity.this, mJobId,
47.                                 taskAttributes, intent);
48.
49.                         Log.i(TAG, "MonitorJob request: " + jrid);
50.                     }
51.                 }
52.             }
53.         }
54.     }
55. }

```

```

45.         }
46.     }
47. }
48.
49. @Override
50. public void onComplete(final String rid, final JobInfo jobInfo) {
51.     Log.i(TAG, "onComplete: Received rid=" + rid);
52.     Log.i(TAG, "JobInfo=" + LogUtil.getJobInfoToString(jobInfo));
53.     if (jobInfo.getJobType() == JobInfo.JobType.COPY) {
54.         CopyJobData copyJobData = jobInfo.getJobData();
55.         if (copyJobData.getJobExecutionMode() == CopyAttributes.JobExecutionMode.ST
ORE) {
56.             mStoredJobId = mJobId;
57.             showSnackBar(getString(R.string.job_stored));
58.         } else {
59.             showSnackBar(getString(R.string.job_completed));
60.         }
61.     }
62. }
63.
64. @Override
65. public void onFail(final String rid, final Result result) {
66.     Log.e(TAG, "onFail: Received rid=" + rid);
67.     showResult(getString(R.string.job_failed), result);
68. }
69.
70. @Override
71. public void onCancel(final String rid) {
72.     Log.i(TAG, "onCancel: Received rid=" + rid);
73.     showSnackBar(getString(R.string.job_cancelled));
74. }
75. }

```

Receiving events to check a copy job state

NOTE: `JobService.monitorJobInForeground()` provides a progress dialog for showing job state by Workpath SDK. If application doesn't have custom dialog, this api will help to provide feature easily. Sample also explains how to call the method.

4.5 Releasing stored Job

Using `CopierService.releaseStoredJob()`, application can request to release stored job. If stored job information like job credentials is mismatched, job will be failed. Refer to `ReleaseStoredTask.java`.

```
1. StoredJobAttributes storedJobAttributes = new StoredJobAttributes.StoredJobBuilder(stor
   edJobId)
2.     .setCopies(copies)
3.     .setJobCredentials(jobCredentials)
4.     .build(caps);
5.
6. // Release the job
7. final String rid = CopierService.releaseStoredJob(activity, storedJobAttributes);
```

Releasing the stored job

4.6 Deleting stored Job

The Workpath SDK provides the API to delete stored job from a device. Sample shows the example in DeleteStoredTask.

```
1. CopierService.deleteStoredJob(activity, storedJobId, jobCredentials, result);
```

Deleting stored job

4.7 Enumerating stored Job

The Workpath SDK provides the API to retrieve the list of stored jobs from a device using `CopierService.enumerateStoredJob()`. Refer to `EnumerateStoredTask.java`.

```
1. @Override
2. protected List<StoredJobInfo> doInBackground(final String... params) {
3.     try {
4.         return CopierService.enumerateStoredJob(mContextRef.get().getActivity(), result
5.     );
6.     } catch (Exception e) {
7.         mErrorMsg = e.getMessage();
8.     }
9.     return null;
10. }
```

Getting the list of stored job

5 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

6 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.